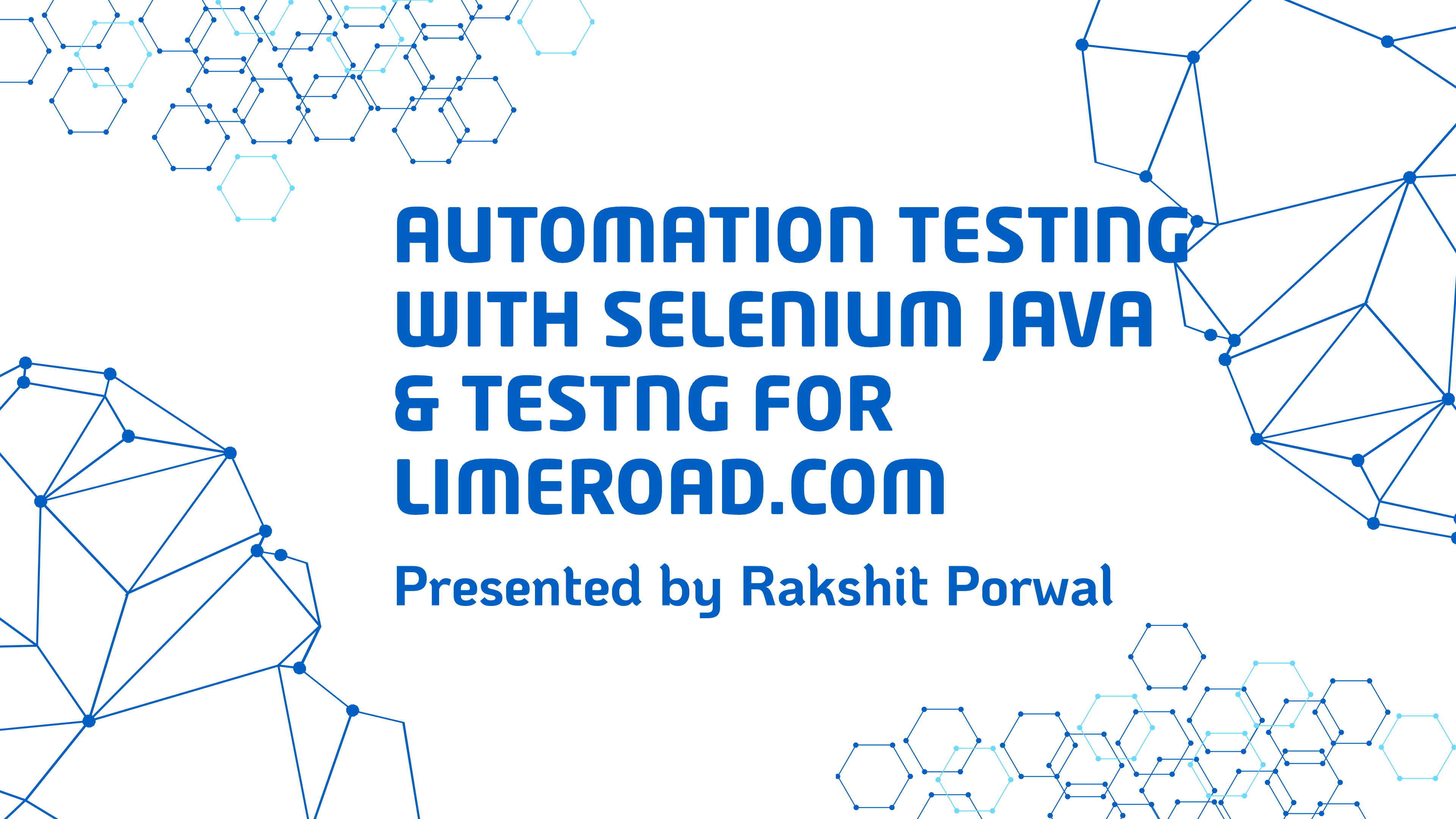




AUTOMATION TESTING WITH SELENIUM JAVA & TESTING FOR LIMEROAD.COM

Presented by Rakshit Porwal

AUTOMATION TESTING WITH SELENIUM JAVA & TESTING FOR LIMEROAD.COM

This training project focuses on automating test cases for limeroad.com to improve testing efficiency and accuracy.

- Tools: Selenium WebDriver, Java, TestNG Framework
- Automates key user flows on limeroad.com
- GitHub integration for version control & collaboration

```
4 Selenium: <:  
9 tritfs {esctan@}- (<  
17 mastTot)  
18v lcr, cAar: (!:  
19 testraglatioo {lo: praectallo}).oket))  
15 tecctinal {sectfenleT):  
17 Test ingCentiotery: {rreat)) (!:  
38 tecstaon Testfnoa: testfol )  
126 har:  
135 ))  
136  
137 {siclent: plao7resst): ((kileit  
138 {osper {scoterleast(test  
139 idehe:  
136
```

AUTOMATION SETUP AND TOOLS



Selenium WebDriver: A powerful browser automation tool that enables interaction with web elements on limeroad.com, simulating real user actions across Chrome, Firefox, and other browsers.



Java & TestNG: Java serves as the programming language for writing robust test scripts. TestNG framework provides annotations (@Test, @BeforeMethod), grouping, parallel execution, and detailed HTML reports.



Environment Setup: ChromeDriver configured for browser-test integration. IDE support via Eclipse or IntelliJ IDEA. Maven for dependency management, with Selenium and TestNG libraries in pom.xml.

KEY TEST CASES IMPLEMENTED



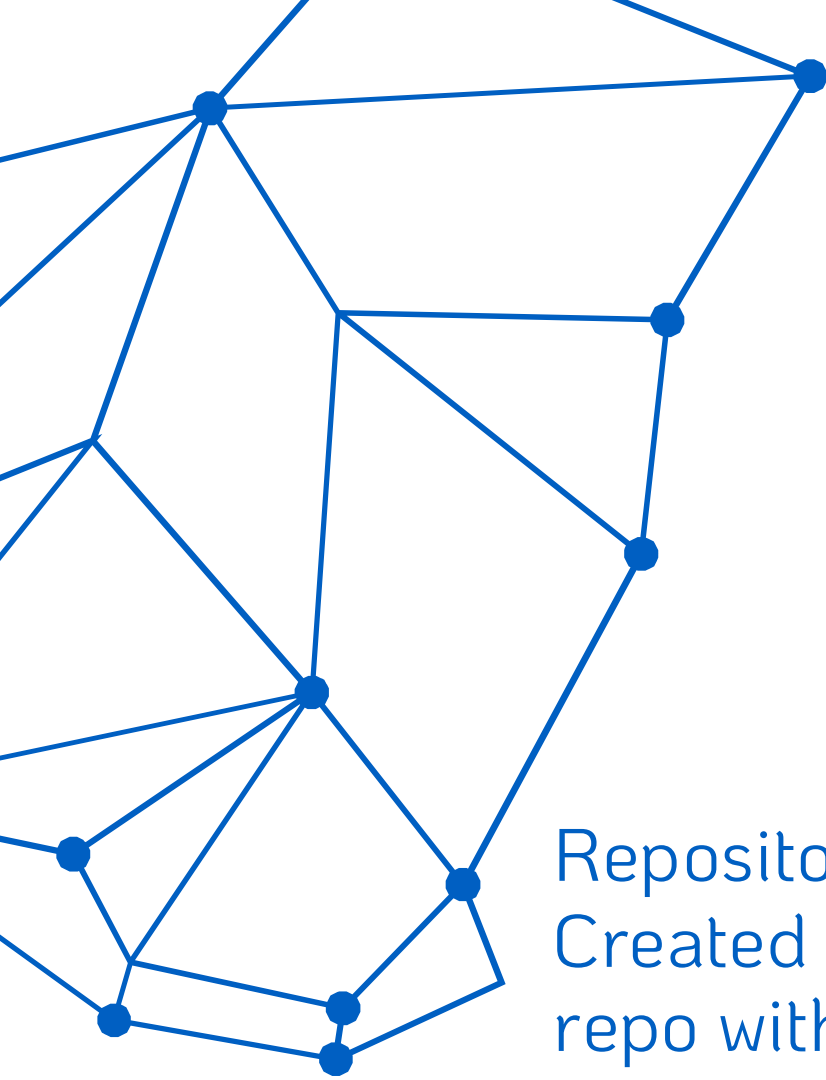
Login Functionality Tests: Automated valid/invalid login scenarios on limeroad.com, verifying successful authentication, error messages, and session handling using TestNG assertions.



Product Search & Filter Validation: Automated search queries and filter selections (category, price, brand) to validate correct results, URL parameters, and page title accuracy.



Add to Cart & Checkout Automation: End-to-end test scripts covering add-to-cart, quantity updates, and checkout flow. TestNG grouping used to organize smoke and regression test suites.



GITHUB INTEGRATION FOR PROJECT MANAGEMENT

Repository Setup

Created a structured GitHub repo with organized folders for test scripts, config files, and reports. Clear README with setup instructions and project overview for easy onboarding.



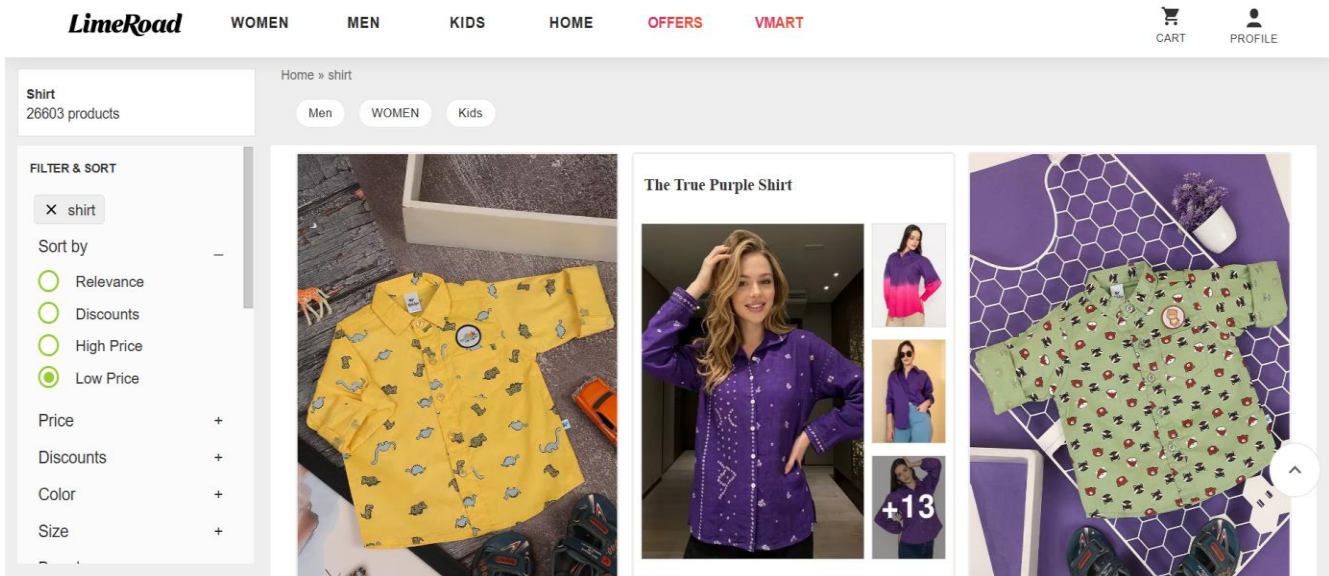
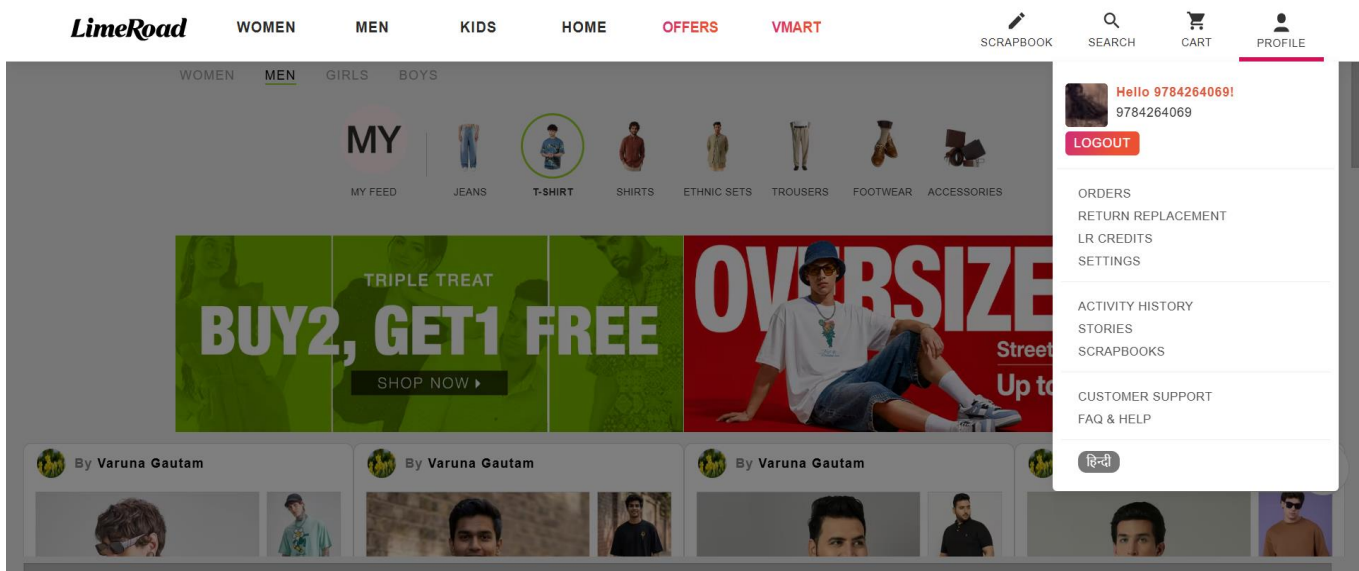
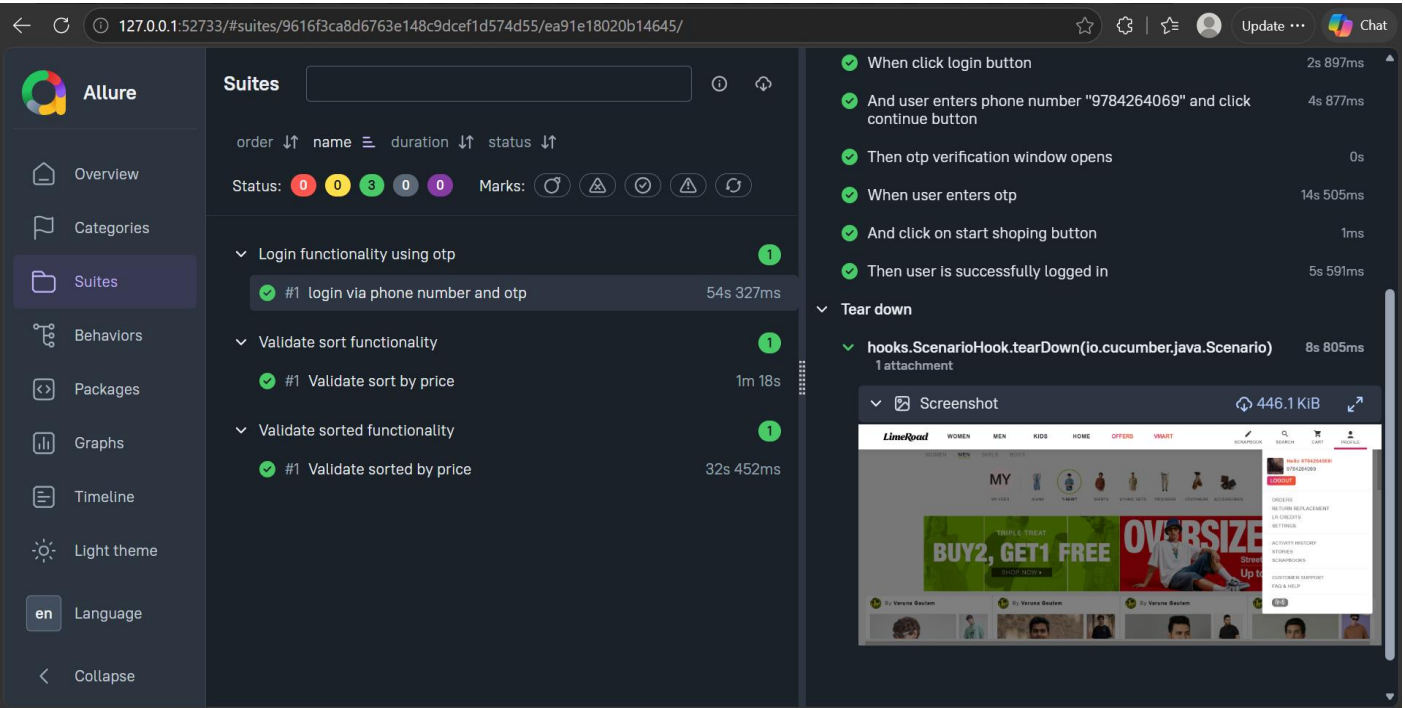
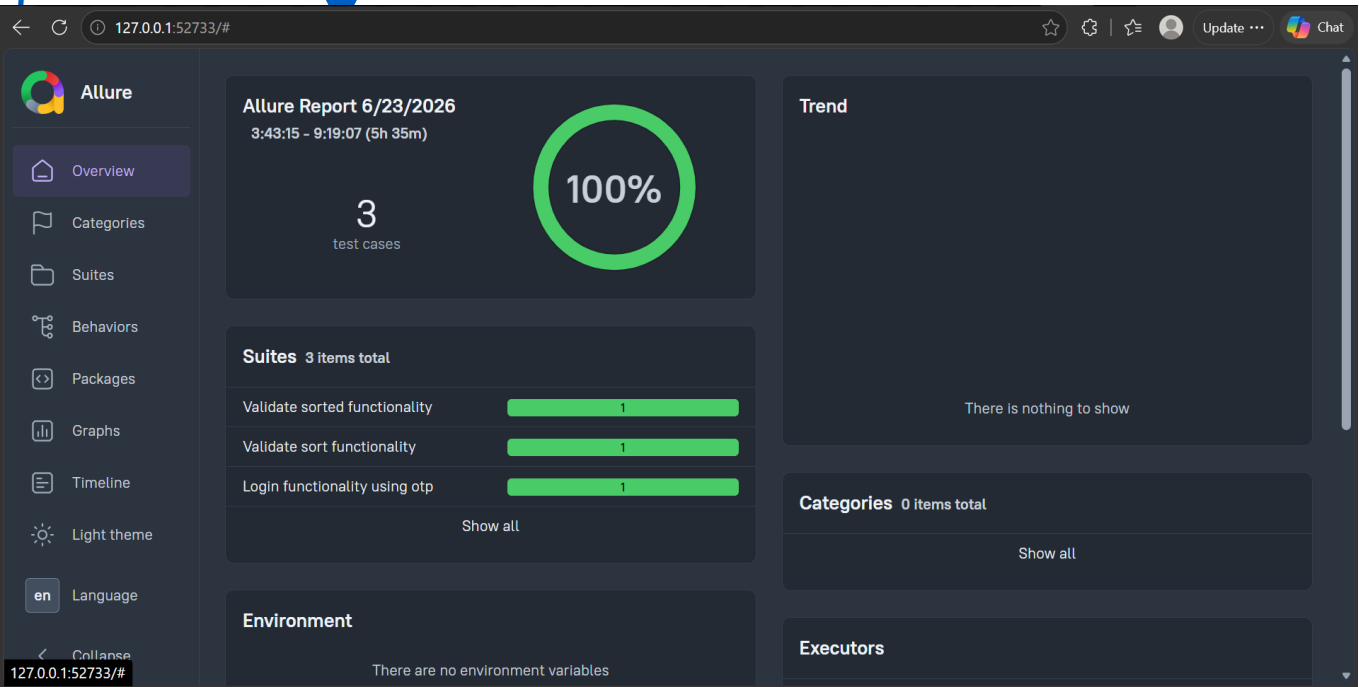
Branch Management
Code pushed to remote repo using Git commands. Feature branches created for each test module. Merging managed via pull requests to maintain code integrity and history.

Collaborative Workflow

Pull requests reviewed by team members before merging. Code reviews ensure quality and consistency. Commit history tracks all changes for full traceability.



Allure Report screenshots

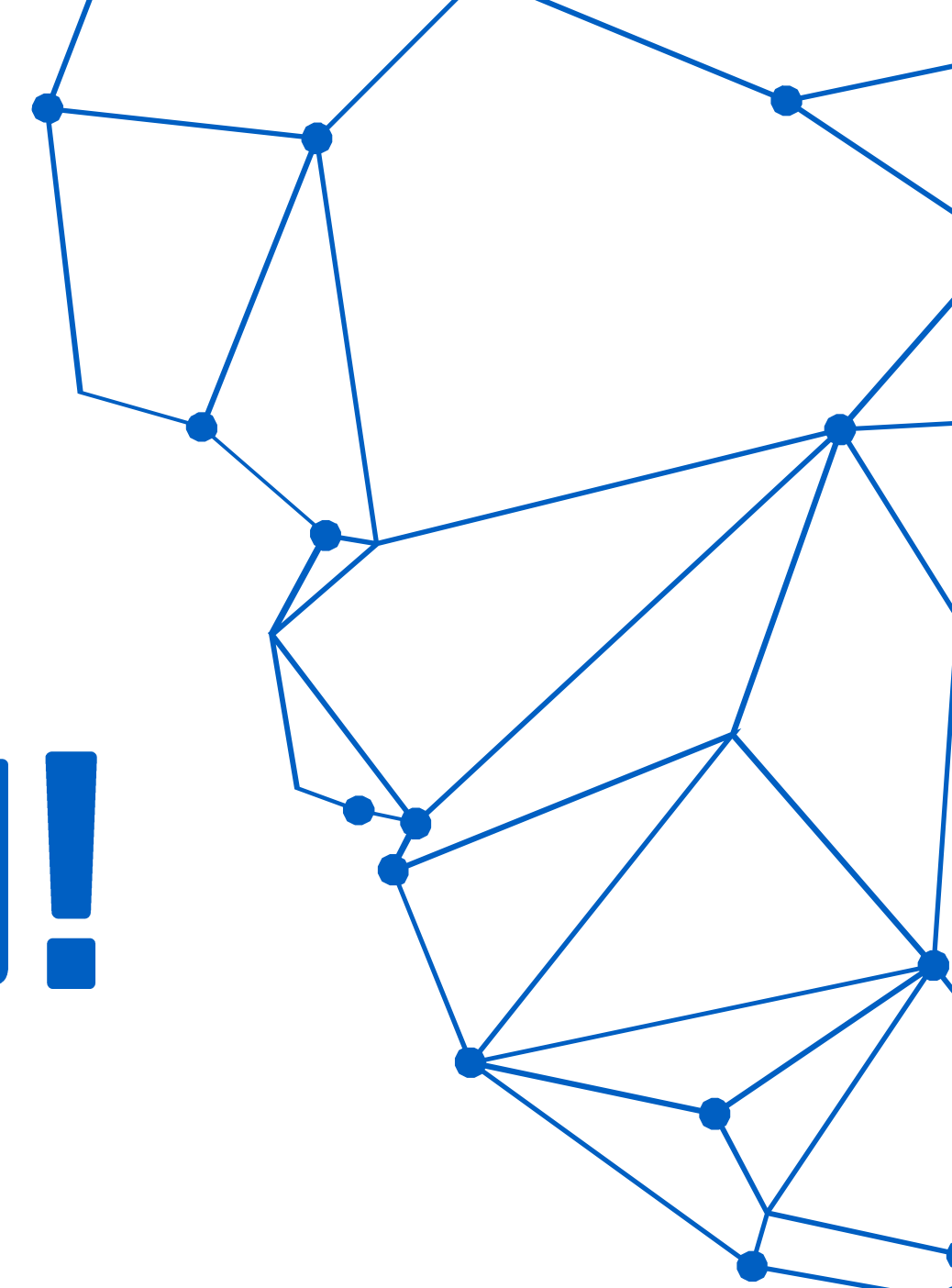
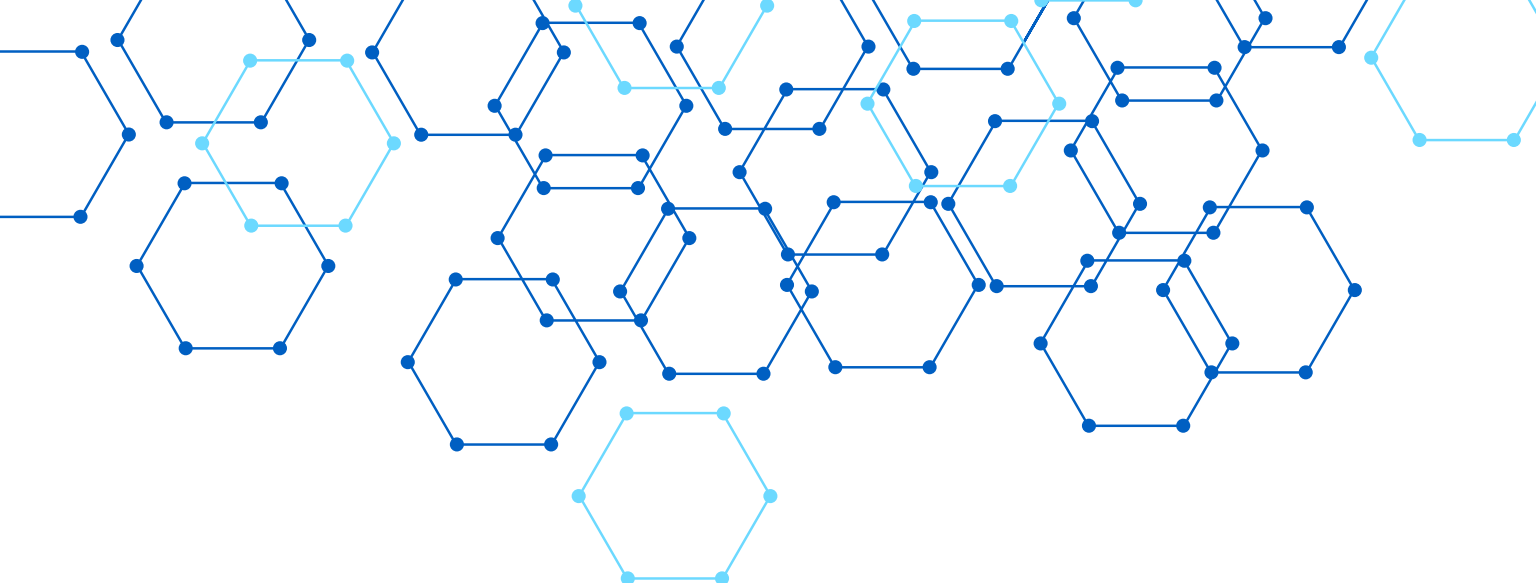


SUMMARY AND NEXT STEPS



Benefits of automation: faster regression testing, improved accuracy across test runs, and reusable scripts that reduce manual effort over time.

Challenges and lessons learned: managing dynamic web elements, handling synchronization issues, and structuring maintainable test code for limeroad.com.



THANK YOU!

Questions & Collaboration Welcome

